Chapter 13. ConfigMaps and Secrets

It's good practice to make container images as reusable as possible. The same image should be able to be used for development, staging, and production. It's even better if the same image is general-purpose enough to be used across applications and services. Testing and versioning are more risky and complicated if images need to be re-created for each new environment. How then do we specialize the use of that image at runtime?

This is where ConfigMaps and Secrets come into play. ConfigMaps are used to provide configuration information for workloads. This can be either fine-grained information like a string or a composite value in the form of a file. Secrets are similar to ConfigMaps but focus on making sensitive information available to the workload. They can be used for things like credentials or TLS certificates.

ConfigMaps

One way to think of a ConfigMap is as a Kubernetes object that defines a small filesystem. Another way is as a set of variables that can be used when defining the environment or command line for your containers. The key thing to note is that the ConfigMap is combined with the Pod right before it is run. This means that the container image and the Pod definition can be reused by many workloads just by changing the ConfigMap that is used.

Creating ConfigMaps

Let's jump right in and create a ConfigMap. Like many objects in Kubernetes, you can create these in an immediate, imperative way, or you can create them from a manifest on disk. We'll start with the imperative method.

First, suppose we have a file on disk (called *my-config.txt*) that we want to make available to the Pod in question, as shown in Example 13-1.

Example 13-1. my-config.txt

```
# This is a sample config file that I might use to configure an application
parameter1 = value1
parameter2 = value2
```

Next, let's create a ConfigMap with that file. We'll also add a couple of simple key/value pairs here. These are referred to as literal values on the command line:

```
$ kubectl create configmap my-config \
  --from-file=my-config.txt \
  --from-literal=extra-param=extra-value \
  --from-literal=another-param=another-value
```

The equivalent YAML for the ConfigMap object we just created is as follows:

```
$ kubectl get configmaps my-config -o yaml

apiVersion: v1
data:
  another-param: another-value
  extra-param: extra-value
  my-config.txt: |
    # This is a sample config file that I might use to configure an application
    parameter1 = value1
    parameter2 = value2
kind: ConfigMap
metadata:
  creationTimestamp: ...
  name: my-config
  namespace: default
  resourceVersion: "13556"
  selfLink: /api/v1/namespaces/default/configmaps/my-config
  uid: 3641c553-f7de-11e6-98c9-06135271a273
```

As you can see, the ConfigMap is just some key/value pairs stored in an object. The interesting part is when you try to *use* a ConfigMap.

Using a ConfigMap

There are three main ways to use a ConfigMap:

Filesystem

You can mount a ConfigMap into a Pod. A file is created for each entry based on the key name. The contents of that file are set to the value.

Environment variable

A ConfigMap can be used to dynamically set the value of an environment variable.

Command-line argument

Kubernetes supports dynamically creating the command line for a container based on ConfigMap values.

Let's create a manifest for `kuard` that pulls all of these together, as shown in Example 13-2.

Example 13-2. kuard-config.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard-config
spec:
  containers:
    - name: test-container
      image: gcr.io/kuar-demo/kuard-amd64:blue
      imagePullPolicy: Always
      command:
        - "/kuard"
        - "$(EXTRA_PARAM)"
      env:
        # An example of an environment variable used inside the container
        - name: ANOTHER_PARAM
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: another-param
        # An example of an environment variable passed to the command to start
        # the container (above).
        - name: EXTRA_PARAM
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: extra-param
      volumeMounts:
```

```
    # Mounting the ConfigMap as a set of files
    - name: config-volume
      mountPath: /config
  volumes:
    - name: config-volume
      configMap:
        name: my-config
  restartPolicy: Never
```

For the filesystem method, we create a new volume inside the Pod and give it the name `config-volume`. We then define this volume to be a ConfigMap volume and point at the ConfigMap to mount. We have to specify where this gets mounted into the `kuard` container with a `volumeMount`. In this case, we are mounting it at `/config`.

Environment variables are specified with a special `valueFrom` member. This references the ConfigMap and the data key to use within that ConfigMap. Command-line arguments build on environment variables. Kubernetes will perform the correct substitution with a special `$(<env-var-name>)` syntax.

Run this Pod, and let's port-forward to examine how the app sees the world:

```
$ kubectl apply -f kuard-config.yaml
$ kubectl port-forward kuard-config 8080
```

Now point your browser to *http://localhost:8080*. We can look at how we've injected configuration values into the program in all three ways. Click the "Server Env" tab on the left. This will show the command line that the app was launched with along with its environment, as shown in Figure 13-1.

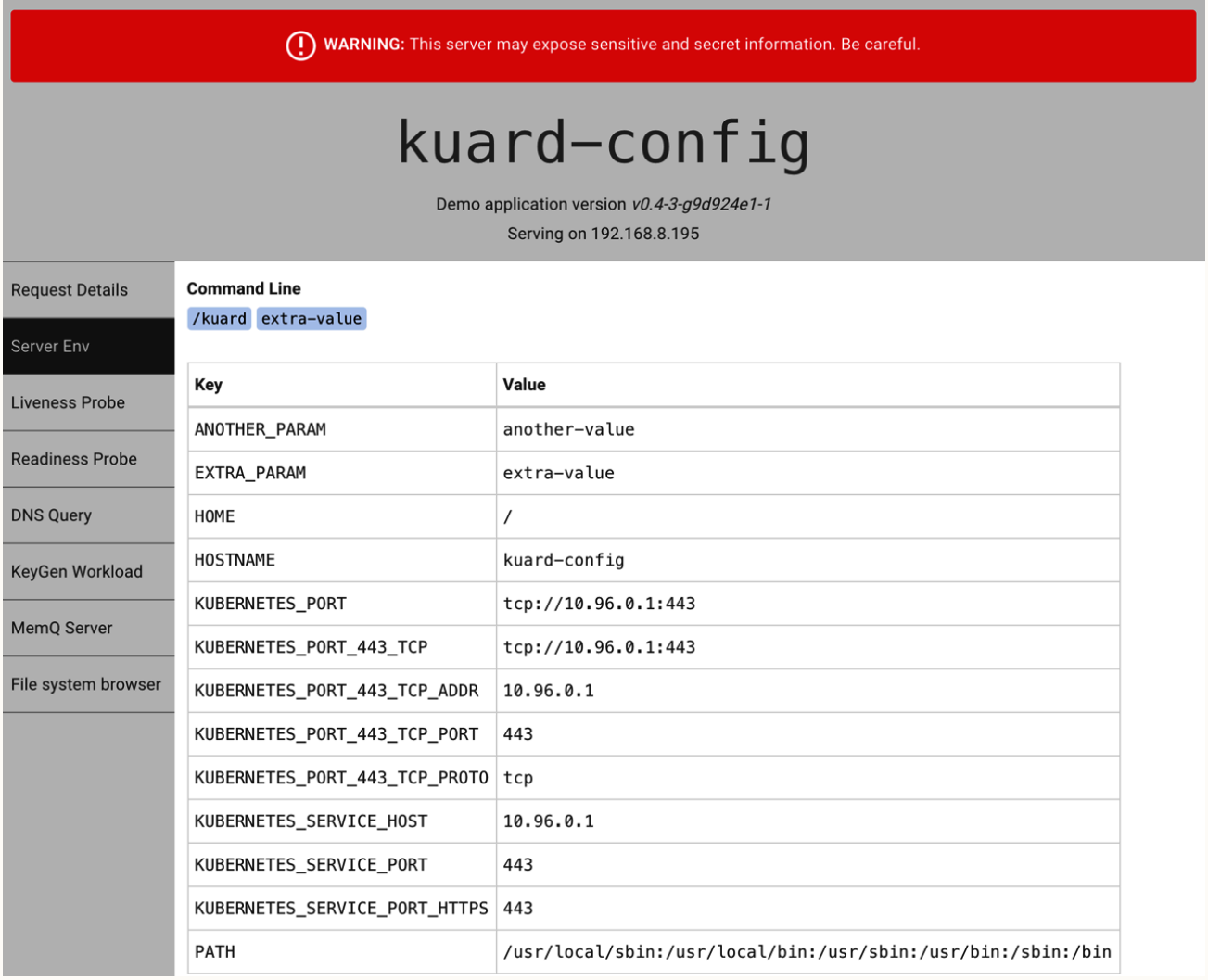


Figure 13-1. `kuard`, showing its environment

Here we can see that we've added two environment variables (`ANOTHER_PARAM` and `EXTRA_PARAM`) whose values are set via the ConfigMap. We've also added an argument to the command line of `kuard` based on the `EXTRA_PARAM` value.

Next, click the "File system browser" tab (Figure 13-2). This lets you explore the filesystem as the application sees it. You should see an entry called `/config`. This is a volume created based on our ConfigMap. If you navigate into that, you'll see that a file has been created for each entry of the ConfigMap. You'll also see some hidden files (prepended with ..) that are used to do a clean swap of new values when the ConfigMap is updated.



Figure 13-2. The /config directory as seen through `kuard`

Secrets

While ConfigMaps are great for most configuration data, there is certain data that is extra sensitive. This includes passwords, security tokens, or other types of private keys. Collectively, we call this type of data "Secrets." Kubernetes has native support for storing and handling this data with care.

Secrets enable container images to be created without bundling sensitive data. This allows containers to remain portable across environments. Secrets are exposed to Pods via explicit declaration in Pod manifests and the Kubernetes API. In this way, the Kubernetes Secrets API provides an application-centric mechanism for exposing sensitive configuration information to applications in a way that's easy to audit and leverages native OS isolation primitives.

The remainder of this section will explore how to create and manage Kubernetes Secrets, and also lay out best practices for exposing Secrets to Pods that require them.

WARNING

By default, Kubernetes Secrets are stored in plain text in the `etcd` storage for the cluster. Depending on your requirements, this may not be sufficient security for you. In particular, anyone who has cluster administration rights in your cluster will be able to read all of the Secrets in the cluster.

In recent versions of Kubernetes, support has been added for encrypting the Secrets with a user-supplied key, generally integrated into a cloud key store. Additionally, most cloud key stores have integration with Kubernetes Secrets Store CSI Driver volumes, enabling you to skip Kubernetes Secrets entirely and rely exclusively on the cloud provider's key store. All of these options should provide you with sufficient tools to craft a security profile that suits your needs.

Creating Secrets

Secrets are created using the Kubernetes API or the `kubectl` command-line tool. Secrets hold one or more data elements as a collection of key/value pairs.

In this section, we will create a Secret to store a TLS key and certificate for the `kuard` application that meets the storage requirements listed previously.

NOTE

The `kuard` container image does not bundle a TLS certificate or key. This allows the `kuard` container to remain portable across environments and distributable through public Docker repositories.

The first step in creating a Secret is to obtain the raw data we want to store. The TLS key and certificate for the `kuard` application can be downloaded by running the following commands:

```
$ curl -o kuard.crt  https://storage.googleapis.com/kuar-demo/kuard.crt
$ curl -o kuard.key https://storage.googleapis.com/kuar-demo/kuard.key
```

WARNING

These certificates are shared with the world and they provide no actual security. Please do not use them except as a learning tool in these examples.

With the *kuard.crt* and *kuard.key* files stored locally, we are ready to create a Secret. Create a Secret named `kuard-tls` using the `create secret` command:

```
$ kubectl create secret generic kuard-tls \
  --from-file=kuard.crt \
  --from-file=kuard.key
```

The `kuard-tls` Secret has been created with two data elements. Run the following command to get details:

```
$ kubectl describe secrets kuard-tls

Name:         kuard-tls
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:         Opaque

Data
====
```

```
kuard.crt:    1050 bytes
kuard.key:    1679 bytes
```

With the `kuard-tls` Secret in place, we can consume it from a Pod by using a Secrets volume.

Consuming Secrets

Secrets can be consumed using the Kubernetes REST API by applications that know how to call that API directly. However, our goal is to keep applications portable. Not only should they run well in Kubernetes, but they should run, unmodified, on other platforms.

Instead of accessing Secrets through the API server, we can use a *Secrets volume*. Secret data can be exposed to Pods using the Secrets volume type. Secrets volumes are managed by the `kubelet` and are created at Pod creation time. Secrets are stored on `tmpfs` volumes (aka RAM disks), and as such are not written to disk on nodes.

Each data element of a Secret is stored in a separate file under the target mount point specified in the volume mount. The `kuard-tls` Secret contains two data elements: *kuard.crt* and *kuard.key*. Mounting the `kuard-tls` Secrets volume to `/tls` results in the following files:

```
/tls/kuard.crt
/tls/kuard.key
```

The Pod manifest in Example 13-3 demonstrates how to declare a Secrets volume, which exposes the `kuard-tls` Secret to the `kuard` container under `/tls`.

Example 13-3. kuard-secret.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard-tls
spec:
  containers:
    - name: kuard-tls
      image: gcr.io/kuar-demo/kuard-amd64:blue
      imagePullPolicy: Always
```

```
    volumeMounts:
    - name: tls-certs
      mountPath: "/tls"
      readOnly: true
  volumes:
    - name: tls-certs
      secret:
        secretName: kuard-tls
```

Create the `kuard-tls` Pod using `kubectl` and observe the log output from the running Pod:

```
$ kubectl apply -f kuard-secret.yaml
```

Connect to the Pod by running:

```
$ kubectl port-forward kuard-tls 8443:8443
```

Now navigate your browser to *https://localhost:8443*. You should see some invalid certificate warnings because this is a self-signed certificate for *kuard.example.com*. If you navigate past this warning, you should see the `kuard` server hosted via HTTPS. Use the "File system browser" tab to find the certificates on disk in the `/tls` directory.

Private Container Registries

A special use case for Secrets is to store access credentials for private container registries. Kubernetes supports using images stored on private registries, but access to those images requires credentials. Private images can be stored across one or more private registries. This presents a challenge for managing credentials for each private registry on every possible node in the cluster.

Image pull Secrets leverage the Secrets API to automate the distribution of private registry credentials. Image pull Secrets are stored just like regular Secrets but are consumed through the `spec.imagePullSecrets` Pod specification field.

Use `kubectl create secret docker-registry` to create this special kind of Secret:

```
$ kubectl create secret docker-registry my-image-pull-secret \
  --docker-username=<username> \
```

```
    --docker-password=<password> \
    --docker-email=<email-address>
```

Enable access to the private repository by referencing the image `pull secret` in the Pod manifest file, as shown in Example 13-4.

Example 13-4. kuard-secret-ips.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard-tls
spec:
  containers:
    - name: kuard-tls
      image: gcr.io/kuar-demo/kuard-amd64:blue
      imagePullPolicy: Always
      volumeMounts:
      - name: tls-certs
        mountPath: "/tls"
        readOnly: true
  imagePullSecrets:
  - name:  my-image-pull-secret
  volumes:
    - name: tls-certs
      secret:
        secretName: kuard-tls
```

If you are repeatedly pulling from the same registry, you can add the Secrets to the default service account associated with each Pod to avoid having to specify the Secrets in every Pod you create.

Naming Constraints

The key names for data items inside of a Secret or ConfigMap are defined to map to valid environment variable names. They may begin with a dot, then are followed by a letter or number, followed by characters including dots, dashes, and underscores. Dots cannot be repeated, and dots and underscores or dashes cannot be adjacent to each other. More formally, this means that they must conform to the regular expression `^[.]?[a-zAZO-9]([.]?[a-zA-ZO-9]+[-_a-zA-ZO-9]?)*$`. Some examples of valid and invalid names for ConfigMaps and Secrets are given in Table 13-1.

Table 13-1. ConfigMap and Secret key examples

Valid key name	Invalid key name
`.auth_token`	`Token..properties`
Key.pem	auth file.json
`config_file`	`_password.txt`

NOTE

When selecting a key name, remember that these keys can be exposed to Pods via a volume mount. Pick a name that is going to make sense when specified on a command line or in a config file. Storing a TLS key as `key.pem` is clearer than `tls-key` when configuring applications to access Secrets.

ConfigMap data values are simple UTF-8 text specified directly in the manifest. Secret data values hold arbitrary data encoded using base64. The use of base64 encoding makes it possible to store binary data. This does, however, make it more difficult to manage Secrets that are stored in YAML files as the base64-encoded value must be put in the YAML. Note that the maximum size for a ConfigMap or Secret is 1 MB.

Managing ConfigMaps and Secrets

ConfigMaps and Secrets are managed through the Kubernetes API. The usual `create`, `delete`, `get`, and `describe` commands work for manipulating these objects.

Listing

You can use the `kubectl get secrets` command to list all Secrets in the current namespace:

```
$ kubectl get secrets

NAME                  TYPE                                  DATA      AGE
default-token-f5jq2   kubernetes.io/service-account-token   3         1h
kuard-tls             Opaque                                2         20m
```

Similarly, you can list all of the ConfigMaps in a namespace:

```
$ kubectl get configmaps

NAME       DATA     AGE
my-config   3       1m
```

`kubectl describe` can be used to get more details on a single object:

```
$ kubectl describe configmap my-config

Name:         my-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
another-param:  13 bytes
extra-param:    11 bytes
my-config.txt:  116 bytes
```

Finally, you can see the raw data (including values in Secrets!) by using a command similar to the following: `kubectl get configmap my-config -o yaml` or `kubectl get secret kuard-tls -o yaml`.

Creating

The easiest way to create a Secret or a ConfigMap is via `kubectl create secret generic` or `kubectl create configmap`. There are a variety of ways to specify the data items that go into the Secret or ConfigMap. These can be combined in a single command:

`--from-file=<filename>`

: Load from the file with the Secret data key that's the same as the filename.

`--from-file=<key>=<filename>`

: Load from the file with the Secret data key explicitly specified.

`--from-file=<directory>`

Load all the files in the specified directory where the filename is an acceptable key name.

`--from-literal=<key>=<value>`

Use the specified key/value pair directly.

Updating

You can update a ConfigMap or Secret and have it reflected in running applications. There is no need to restart if the application is configured to reread configuration values. Next, we will describe three ways to update ConfigMaps or Secrets.

Update from file

If you have a manifest for your ConfigMap or Secret, you can just edit it directly and replace it with a new version using `kubectl replace -f <filename>`. You can also use `kubectl apply -f <filename>` if you previously created the resource with `kubectl apply`.

Due to the way that datafiles are encoded into these objects, updating a configuration can be a bit cumbersome; there is no `kubectl` command that supports loading data from an external file. The data must be stored directly in the YAML manifest.

The most common use case is when the ConfigMap is defined as part of a directory or list of resources and everything is created and updated together. Oftentimes these manifests will be checked into source control.

WARNING

It is generally a bad idea to check Secret YAML files into source control because it is too easy to inadvertently push these files someplace public and leak your Secrets.

Re-create and update

If you store the inputs into your ConfigMaps or Secrets as separate files on disk (as opposed to embedded into YAML directly), you can use `kubectl` to re-create the manifest and then use it to update the object, which will look something like this:

```
$ kubectl create secret generic kuard-tls \
  --from-file=kuard.crt --from-file=kuard.key \
```

```
--dry-run -o yaml | kubectl replace -f -
```

This command line first creates a new Secret with the same name as our existing Secret. If we just stopped there, the Kubernetes API server would return an error complaining that we are trying to create a Secret that already exists. Instead, we tell `kubectl` not to actually send the data to the server but instead to dump the YAML that it *would have* sent to the API server to `stdout`. We then pipe that to `kubectl replace` and use `-f -` to tell it to read from `stdin`. In this way, we can update a Secret from files on disk without having to manually base64-encode data.

Edit current version

The final way to update a ConfigMap is to use `kubectl edit` to bring up a version of the ConfigMap in your editor so you can tweak it (you could also do this with a Secret, but you'd be stuck managing the base64 encoding of values on your own):

```
$ kubectl edit configmap my-config
```

You should see the ConfigMap definition in your editor. Make your desired changes and then save and close your editor. The new version of the object will be pushed to the Kubernetes API server.

Live updates

Once a ConfigMap or Secret is updated using the API, it'll be automatically pushed to all volumes that use that ConfigMap or Secret. It may take a few seconds, but the file listing and contents of the files, as seen by `kuard`, will be updated with these new values. Using this live update feature, you can update the configuration of applications without restarting them.

Currently there is no built-in way to signal an application when a new version of a ConfigMap is deployed. It is up to the application (or some helper script) to look for the config files to change and reload them.

Using the file browser in `kuard` (accessed through `kubectl port-forward`) is a great way to interactively play with dynamically updating Secrets and ConfigMaps.

Summary

ConfigMaps and Secrets are a great way to provide dynamic configuration in your application. They allow you to create a container image (and Pod definition) once and reuse it in different contexts. This can include using the exact same image as you move from development to staging to production. It can also include using a single image across multiple teams and services. Separating configuration from application code will make your applications more reliable and reusable.

Previous chapter
< 12. Jobs

Next chapter
14. Role-Based Access Control for Kubernetes >